Types of Variables

- *Based on the type of value represented by the variable, all variables are divided into 2 types. They are:*
    1) *Primitive variables*
    2) *Reference variables*

Primitive variables: *Primitive variables can be used to represent primitive values.*

Example: *int x=10;*

Reference variables: *Reference variables can be used to refer to objects.*

Example: *Student s=new Student();*

Diagram:



- *Based on the purpose and position of the declaration all variables are divided into the following 3 types.*
    1) *Instance variables*
    2) *Static variables*
    3) *Local variables*

Instance variables:

- *If the value of a variable is varied from object to object, such type of variables are called instance variables.*
- *For every object, a separate copy of the instance variables will be created.*
- *Instance variables will be created at the time of object creation and destroyed at the time of object destruction; hence, the scope of instance variables is exactly the same as the scope of objects.*
- *Instance variables will be stored on the heap as part of the object.*
- *Instance variables should be declared within the class directly, but outside of any method or block or constructor.*
- *Instance variables can be accessed directly from the Instance area. But it cannot be accessed directly from the static area.*
- *But by using object reference, we can access instance variables from a static area.*

Example:

```
class Test
{
    int i=10;
    public static void main(String[] args)
    {
        //System.out.println(i);//C.E:non-static variable i cannot be referenced from a static context(invalid)
        Test t=new Test();
        System.out.println(t.i);//10(valid)
        t.methodOne();
    }
    public void methodOne()
    {
```

Abhishek Kr. Tiwari Java Notes

```
        System.out.println(i);//10(valid)
    }
}
```

- *For the instance variables it is not required to perform initialization JVM will always provide default values.*

Example:

```
class Test
{
    boolean b;
    public static void main(String[] args)
    {
        Test t=new Test();
        System.out.println(t.b);//false
    }
}
```

- *Instance variables are also known as object-level variables or attributes.*

Static variables:

- *If the value of a variable is not varied from object to object, such of variable is not recommended to be declared as an instance variable. We have to declare such of variable at the class level by using the static modifier.*
- *In the case of instance variables for every object, a separate copy will be created, but in the case of static variables for the entire class, only one copy will be created and shared by every object of that class.*
- *Static variables will be created at the time of class loading and destroyed at the time of class unloading, hence the scope of the static variable is exactly the same as the scope of the* ***.class*** *file.*
- *Static variables will be stored in the method area. Static variables should be declared within the class directly, but outside of any method or block or constructor.*
- *Static variables can be accessed from both instance and static areas directly.*
- *We can access static variables either by class name or by object reference, but the usage of class name is recommended.*
- *But within the same class, it is not required to use class name; we can access it directly.*

1) *Start JVM.*
2) *Create and start the Main Thread by the JVM.*
3) *Locate(find) Test.class by the main Thread.*
4) *Load Test.class by main Thread.*
5) *Execution of main() method.*
6) *Unload Test.class*
7) *Terminate the main Thread.*
8) *Shutdown JVM.*

Example:

```
class Test
{
```

Abhishek Kr Tiwari Java Notes

```
    static int i=10;
    public static void main(String[] args)
    {
        Test t=new Test();
        System.out.println(t.i);//10
        System.out.println(Test.i);//10
        System.out.println(i);//10
    }
}
```

- *For the static variables, it is not required to perform initialisation explicitly; the JVM will always provide default values.*

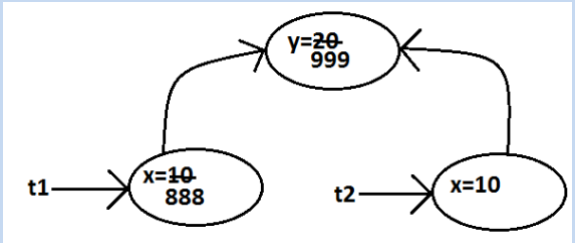
Example:

```
class Test
{
    static String s;
    public static void main(String[] args)
    {
        System.out.println(s);//null
    }
}
```

Example:

```
class Test
{
    int x=10;
    static int y=20;
    public static void main(String[] args)
    {
        Test t1=new Test();
        t1.x=888;
        t1.y=999;
        Test t2=new Test();
        System.out.println(t2.x+"----"+t2.y);//10----999
    }
}
```

Diagram:



- *Static variables are also known as class-level variables or fields.*

Local variables:

Abhishek Kr. Tiwari Java Notes

- *Sometimes, to meet the temporary requirements of the programmer, we can declare variables inside a method or block or a constructor. Such types of variables are called local variables or automatic variables or temporary variables, or stack variables.*
- *The local variables will be created as part of the block execution in which it is declared and destroyed once that block execution completes. Hence, the scope of the local variables is exactly the same as the scope of the block in which we declared them.*

Example 1:

```
class Test
{
    public static void main(String[] args)
    {
        int i=0;
        for(int j=0;j<3;j++)
        {
            i=i+j;
        }
        System.out.println(i+"----"+j);
    }
}
```

C.E

```
javac Test.java
Test.java:10: cannot find symbol
symbol  : variable j
location: class Test
```

Example 2:

```
class Test
{
    public static void main(String[] args)
    {
        try
        {
            int i=Integer.parseInt("ten");
        }
        catch(NullPointerException e)
        {
            System.out.println(i);
        }
    }
}
```

C.E

```
javac Test.java
Test.java:11: cannot find symbol
symbol  : variable i
location: class Test
```

- *The local variables will be stored on the stack.*
- *For the local variables JVM won't provide any default values compulsory we should perform initialisation explicitly before using that variable.*

Abhishek Kr. Tiwari Java Notes

Example:

```
class Test
{
    public static void main(String[] args)
    {
        int x;
        System.out.println("hello");//hello
    }
}
```

```
class Test
{
    public static void main(String[] args)
    {
        int x;
      System.out.println(x);//C.E:variable x might not have been initialized
    }
}
```

Example:

```
class Test
{
    public static void main(String[] args)
    {
            int x;
            if(args.length>0)
            {
                x=10;
            }
            System.out.println(x);//C.E:variable x might not have been initialized
    }
}
```

Example:

```
class Test
{
    public static void main(String[] args)
    {
            int x;
            if(args.length>0)
            {
                x=10;
            }
            else
            {
                x=20;
            }
            System.out.println(x);
    }
}
```

Output:

```
java Test x
10
java Test x y
10
java Test
```

Abhishek Kr. Tiwari Java Notes

- *It is never recommended to perform initialisation for the local variables inside logical blocks because there is no guarantee of executing that block always at runtime.*
- *It is highly recommended to perform initialisation for the local variables at the time of declaration at least with default values.*

Note: *The only applicable modifier for local variables is final. If we are using any other modifier, we will get compile-time error.*

Example:

```
class Test
{
    public static void main(String[] args)
    {

        public int x=10;
        private int x=10;
        protected int x=10;   } C.E: illegal start of expression
        static int x=10;
        volatile int x=10;
        transient int x=10;
        final int x=10;//(valid)
    }
}
```

Conclusions:

1) *For the static and instance variables, it is not required to perform initialisation explicitly JVM will provide default values. But for the local variables JVM won't provide any default values compulsory we should perform initialisation explicitly before using that variable.*
2) *For every object, a separate copy of the instance variable will be created, whereas for the entire class, a single copy of the static variable will be created. For every Thread, a separate copy of the local variable will be created.*
3) *Instance and static variables can be accessed by multiple Threads simultaneously, and hence these are not thread-safe, but local variables can be accessed by only one Thread at a time, and hence local variables are thread-safe.*

Abhishek Kr. Tiwari Java Notes